Indian Institute of Technology Dharwad
CS201L: Artificial Intelligence Laboratory

Lab 7: Logistic Regression and Support Vector Machine

February 19, 2026

1 Problem Statement

You are given a real-world multiclass classification dataset as a CSV file named `activity-detection.csv`, constructed by combining the original training and testing files of the Human Activity Recognition (HAR) using Smartphones Dataset.

The objective is to classify human activities into one of six daily activities based on sensor-based features extracted from smartphone inertial signals.

2 Dataset Description

The Human Activity Recognition database was built from recordings of 30 participants performing activities of daily living (ADL) while carrying a waist-mounted smartphone with embedded inertial sensors.

Each subject performed six activities:

- WALKING
- WALKING UPSTAIRS
- WALKING DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The smartphone (Samsung Galaxy S II) captured:

- 3-axial linear acceleration (accelerometer)
- 3-axial angular velocity (gyroscope)

at a constant sampling rate of 50Hz. The signals were preprocessed using noise filtering and segmented into fixed-width sliding windows of 2.56 seconds with 50% overlap (128 readings per window).

From each window, a vector of features was extracted from both time and frequency domains.

2.1 Dataset Properties

- Total subjects: 30
- Activities: 6 (multiclass)
- Total features: 561 (numerical)
- Feature domains: Time-domain and Frequency-domain
- Sensors: Accelerometer, Gyroscope
- Target attribute: Activity
- Data format: CSV file
- Dataset file: `activity-detection.csv`

2.2 Attribute Information

For each record in the dataset, the following information is provided:

- Triaxial acceleration from the accelerometer (total acceleration)
- Triaxial body acceleration (body motion component)
- Triaxial angular velocity from the gyroscope
- Gravity acceleration components
- 561-feature vector extracted from time and frequency domains
- Activity label (multiclass target)
- Subject identifier (ID of participant)

2.3 Dataset Statistics

Table 1: Overall Dataset Information

Property	**Value**
Total Samples	10,299
Total Features	561
Target Classes	6
Missing Values	0
Data Types	float64 (561), int64 (1), object (1)

3 Dataset Files

We are providing you with the prepared datasets (train, validation, and test datasets) in the CSV files.

Table 2: Activity-wise Sample Distribution

Activity	Sample Count	Percentage (%)
LAYING	1,944	18.88
STANDING	1,906	18.51
SITTING	1,777	17.25
WALKING	1,722	16.72
WALKING UPSTAIRS	1,544	14.99
WALKING DOWNSTAIRS	1,406	13.65
Total	**10,299**	**100.00**

3.1 Original Dataset

- Train data: `activity_train.csv`
- Validation data: `activity_validation.csv`
- Test data: `activity_test.csv`

3.2 Standardized Dataset

- Train data: `activity_scaled_train.csv`
- Validation data: `activity_scaled_validation.csv`
- Test data: `activity_scaled_test.csv`

3.3 PCA Transformed Dataset (All Components)

- Train data: `activity_pcaall_train.csv`
- Validation data: `activity_pcaall_validation.csv`
- Test data: `activity_pcaall_test.csv`

3.4 PCA Transformed Dataset (99% Variance)

- Train data: `activity_pca99_train.csv`
- Validation data: `activity_pca99_validation.csv`
- Test data: `activity_pca99_test.csv`

4 Tasks

Create a **separate Jupyter Notebook for each dataset** as described in the four parts below. Within each notebook, apply all four classification methods and compare their results at the end of the notebook.

4.1 Notebook 1: Original data

Load the train, validation, and test data from `activity_train.csv`, `activity_validation.csv`, and `activity_test.csv` respectively for all tasks in this notebook.

4.1.1 Task 1.1: Logistic regression

1. Implement Logistic Regression classification using `sklearn.linear_model.LogisticRegression`. Set the `solver` hyperparameter to `liblinear`, which internally follows a One-vs-Rest strategy for multiclass classification, meaning it trains a separate binary classifier for each activity class against all remaining classes.
2. Train the model on the training data and predict class labels for the validation data and the test data.
3. Evaluate and report the following performance metrics on both the validation data and the test data:
    - Confusion matrix
    - Accuracy, Precision, Recall, and F1-score

4.1.2 Task 1.2: SVM with a Linear kernel

1. Implement SVM with a linear kernel using `sklearn.svm.SVC(kernel='linear')` with the default regularization parameter C = 1
2. Train the model on the training data and predict class labels for the validation data and the test data.
3. Evaluate and report the following performance metrics on both the validation data and the test data:
    - Confusion matrix
    - Accuracy, Precision, Recall, and F1-score

4.1.3 Task 1.3: SVM with a Polynomial kernel

1. Implement SVM with a polynomial kernel using `sklearn.svm.SVC(kernel='poly')` with the default regularization parameter C = 1 and the default kernel parameter γ.
2. Train models using degree values of 2, 3, 4, and 5. For each degree, predict class labels for the **validation data** and record the validation accuracy.
3. Select the degree that yields the highest accuracy on the validation data as the best degree. Using this best degree evaluate it on the **test data**.
4. Report the validation accuracy for all four degree values. For the best-degree model, evaluate and report the following performance metrics on the test data:
    - Confusion matrix
    - Accuracy, Precision, Recall, and F1-score

4.1.4 Task 1.4: SVM with a Gaussian (RBF) kernel

1. Implement SVM with a Gaussian (RBF) kernel using `sklearn.svm.SVC(kernel='rbf')` with the default regularization parameter C= 1 and the default kernel parameter γ.
2. Train the model on the training data and predict class labels for the validation data and the test data.
3. Evaluate and report the following performance metrics on both the validation data and the test data:
   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.1.5 Task 1.5: Comparison on Original Data

Summarize and compare the accuracy of all four classifiers on the original dataset in the form of a table (separately for validation and test data). Briefly discuss which classifier performed best.

4.2 Notebook 2: Scaled Data

Load the train, validation, and test data from `activity_scaled_train.csv`, `activity_scaled_validation.csv`, and `activity_scaled_test.csv` respectively for all tasks in this notebook.

4.2.1 Task 2.1: Logistic regression

1. Implement Logistic Regression classification using `sklearn.linear_model.LogisticRegression`. Set the `solver` hyperparameter to `liblinear`, which internally follows a One-vs-Rest strategy for multiclass classification.
2. Train the model on the training data and predict class labels for the validation data and the test data.
3. Evaluate and report the following performance metrics on both the validation data and the test data:
   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.2.2 Task 2.2: SVM with a Linear kernel

1. Implement SVM with a linear kernel using `sklearn.svm.SVC(kernel='linear')` with the default regularization parameter C = 1.
2. Train the model on the training data and predict class labels for the validation data and the test data.

3. Evaluate and report the following performance metrics on both the validation data and the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.2.3 Task 2.3: SVM with a Polynomial kernel

1. Implement SVM with a polynomial kernel using `sklearn.svm.SVC(kernel='poly')` with the default regularization parameter C = 1 and the default kernel parameter γ.

2. Train models using degree values of 2, 3, 4, and 5. For each degree, predict class labels for the **validation data** and record the validation accuracy.

3. Select the degree that yields the highest accuracy on the validation data as the best degree. Train a final model using this best degree and evaluate it on the **test data**.

4. Report the validation accuracy for all four degree values. For the best-degree model, evaluate and report the following performance metrics on the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.2.4 Task 2.4: SVM with a Gaussian (RBF) kernel

1. Implement SVM with a Gaussian (RBF) kernel using `sklearn.svm.SVC(kernel='rbf')` with the default regularization parameter C = 1 and the default kernel parameter γ.

2. Train the model on the training data and predict class labels for the validation data and the test data.

3. Evaluate and report the following performance metrics on both the validation data and the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.2.5 Task 2.5: Comparison on Scaled Data

Summarize and compare the accuracy of all four classifiers on the scaled dataset in the form of a table (separately for validation and test data). Briefly discuss which classifier performed best and.

4.3 Notebook 3: PCA Transformed Data (All Components)

Load the train, validation, and test data from `activity_pcaall_train.csv`, `activity_pcaall_validation.csv`, and `activity_pcaall_test.csv` respectively for all tasks in this notebook.

4.3.1 Task 3.1: Logistic regression

1. Implement Logistic Regression classification using `sklearn.linear_model.LogisticRegression`. Set the `solver` hyperparameter to `liblinear`, which internally follows a One-vs-Rest strategy for multiclass classification.

2. Train the model on the training data and predict class labels for the validation data and the test data.

3. Evaluate and report the following performance metrics on both the validation data and the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.3.2 Task 3.2: SVM with a Linear kernel

1. Implement SVM with a linear kernel using `sklearn.svm.SVC(kernel='linear')` with the default regularization parameter C = 1.

2. Train the model on the training data and predict class labels for the validation data and the test data.

3. Evaluate and report the following performance metrics on both the validation data and the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.3.3 Task 3.3: SVM with a Polynomial kernel

1. Implement SVM with a polynomial kernel using `sklearn.svm.SVC(kernel='poly')` with the default regularization parameter C = 1 and the default kernel parameter γ.

2. Train models using degree values of 2, 3, 4, and 5. For each degree, predict class labels for the **validation data** and record the validation accuracy.

3. Select the degree that yields the highest accuracy on the validation data as the best degree. Train a final model using this best degree and evaluate it on the **test data**.

4. Report the validation accuracy for all four degree values. For the best-degree model, evaluate and report the following performance metrics on the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.3.4 Task 3.4: SVM with a Gaussian (RBF) kernel

1. Implement SVM with a Gaussian (RBF) kernel using `sklearn.svm.SVC(kernel='rbf')` with the default regularization parameter C = 1 and the default kernel parameter γ.
2. Train the model on the training data and predict class labels for the validation data and the test data.
3. Evaluate and report the following performance metrics on both the validation data and the test data:
   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.3.5 Task 3.5: Comparison on PCA All components data

Summarize and compare the accuracy of all four classifiers on the PCA all-components dataset in the form of a table (separately for validation and test data). Briefly discuss which classifier performed best .

4.4 Notebook 4: PCA Transformed Data (99% Variance)

Load the train, validation, and test data from `activity_pca99_train.csv`, `activity_pca99_validation.csv`, and `activity_pca99_test.csv` respectively for all tasks in this notebook.

4.4.1 Task 4.1: Logistic regression

1. Implement Logistic Regression classification using `sklearn.linear_model.LogisticRegression`. Set the `solver` hyperparameter to `liblinear`, which internally follows a One-vs-Rest strategy for multiclass classification.
2. Train the model on the training data and predict class labels for the validation data and the test data.
3. Evaluate and report the following performance metrics on both the validation data and the test data:
   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.4.2 Task 4.2: SVM with a Linear kernel

1. Implement SVM with a linear kernel using `sklearn.svm.SVC(kernel='linear')` with the default regularization parameter C = 1.
2. Train the model on the training data and predict class labels for the validation data and the test data.

3. Evaluate and report the following performance metrics on both the validation data and the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.4.3 Task 4.3: SVM with a Polynomial kernel

1. Implement SVM with a polynomial kernel using `sklearn.svm.SVC(kernel='poly')` with the default regularization parameter C = 1 and the default kernel parameter γ.
2. Train models using degree values of 2, 3, 4, and 5. For each degree, predict class labels for the **validation data** and record the validation accuracy.
3. Select the degree that yields the highest accuracy on the validation data as the best degree. Train a final model using this best degree and evaluate it on the **test data**.
4. Report the validation accuracy for all four degree values. For the best-degree model, evaluate and report the following performance metrics on the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.4.4 Task 4.4: SVM with a Gaussian (RBF) kernel

1. Implement SVM with a Gaussian (RBF) kernel using `sklearn.svm.SVC(kernel='rbf')` with the default regularization parameter C = 1 and the default kernel parameter γ.
2. Train the model on the training data and predict class labels for the validation data and the test data.
3. Evaluate and report the following performance metrics on both the validation data and the test data:

   - Confusion matrix
   - Accuracy, Precision, Recall, and F1-score

4.4.5 Task 4.5: Comparison on PCA 99% Variance Data

Summarize and compare the accuracy of all four classifiers on the PCA 99% variance dataset in the form of a table (separately for validation and test data). Briefly discuss which classifier performed best .

4.5 Overall Comparison Across All Datasets

At the end of **Notebook 4**, consolidate the accuracy results from all four notebooks and present a comprehensive comparison table as shown in the sample code below.

```
# Define the table data
data = {
    "Dataset": ["Original", "Scaled", "PCA All", "PCA 99"],
    "Logistic Regression": [acc_lr_orig, acc_lr_scaled,
                            acc_lr_pcaall, acc_lr_pca99],
    "SVM Linear":          [acc_lin_orig, acc_lin_scaled,
                            acc_lin_pcaall, acc_lin_pca99],
    "SVM Polynomial":      [acc_poly_orig, acc_poly_scaled,
                            acc_poly_pcaall, acc_poly_pca99],
    "SVM Gaussian":        [acc_rbf_orig, acc_rbf_scaled,
                            acc_rbf_pcaall, acc_rbf_pca99]
}

# Create DataFrame
df = pd.DataFrame(data)
df.set_index("Dataset", inplace=True)

# Print DataFrame
print(f"Validation Accuracy Comparison\n{df}")
print(f"\nTest Accuracy Comparison\n{df}")
```

5 Notes

1. Create **four separate Jupyter Notebooks**, one per dataset: Original Data (Notebook 1), Scaled Data (Notebook 2), PCA All Components (Notebook 3), and PCA 99% Variance (Notebook 4). All classifiers for a given dataset must be implemented within the same notebook.

2. For Logistic Regression, set the `solver` hyperparameter to `liblinear`. This solver natively follows a One-vs-Rest approach for multiclass problems, training one binary classifier per class. Note that the `multi_class` parameter has been deprecated in recent versions of scikit-learn and should not be used.

   ```
   from sklearn.linear_model import LogisticRegression
   logistic_reg = LogisticRegression(solver='liblinear',
                                     max_iter=1000)
   logistic_reg.fit(X_train, y_train)
   y_val_pred  = logistic_reg.predict(X_val)
   y_test_pred = logistic_reg.predict(X_test)
   ```

3. For SVM with a linear kernel, use the following:

   ```
   from sklearn.svm import SVC
   linear_svm = SVC(kernel='linear', C=1.0)
   linear_svm.fit(X_train, y_train)
   y_val_pred  = linear_svm.predict(X_val)
   y_test_pred = linear_svm.predict(X_test)
   ```

4. For SVM with a Gaussian (RBF) kernel, use the following:

```
from sklearn.svm import SVC
rbf_svm = SVC(kernel='rbf', C=1.0, gamma='scale')
rbf_svm.fit(X_train, y_train)
y_val_pred  = rbf_svm.predict(X_val)
y_test_pred = rbf_svm.predict(X_test)
```

5. For SVM with a polynomial kernel, use the validation set to select the best degree from $\{2, 3, 4, 5\}$, then report test metrics only for the best degree:

```
from sklearn.svm import SVC

best_degree, best_val_acc = None, 0
for degree in [2, 3, 4, 5]:
    poly_svm = SVC(kernel='poly', degree=degree,
                   C=1.0, gamma='scale')
    poly_svm.fit(X_train, y_train)
    val_acc = poly_svm.score(X_val, y_val)
    print(f"Degree={degree}, Validation Accuracy={val_acc:.4f}")
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_degree  = degree

print(f"\nBest Degree: {best_degree}")
best_poly_svm = SVC(kernel='poly', degree=best_degree,
                    C=1.0, gamma='scale')
best_poly_svm.fit(X_train, y_train)
y_test_pred = best_poly_svm.predict(X_test)
```

6. You can import `confusion_matrix`, `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, and `classification_report` from `sklearn.metrics`. Refer to the slides uploaded on Moodle for guidance on computing and interpreting these metrics.

7. Each notebook must end with a dataset-wise comparison table (Tasks 1.5, 2.5, 3.5, 4.5) comparing the accuracy of all four classifiers on that dataset. The overall cross-dataset comparison is to be placed at the end of Notebook 4.

6 Submission Instructions

Please upload the completed Jupyter Notebooks. Copy all Jupyter Notebook and data files into a folder and rename it as `your_rollnumber_Lab7`, then create a zip file named `your_rollnumber_Lab7.zip`. Finally, upload the zip file on Moodle for evaluation.